API World | KEYNOTE

Justifying AI Without Sacrifice of Functionality

Why the most-requested feature in software is also the easiest one to get wrong and the practical frameworks that keep AI honest.

redgate | About me

Kellyn Gorman

Multiplatform and AI Advocate

https://dbakevlar.com
https://redgate.com



Story Time

Website for volunteer organization

Admin panel now has AI built in, supporting most features

Capable of 50% of requests

Solutions are often secondary “hero” layers on top of traditional page builds.

The Challenge



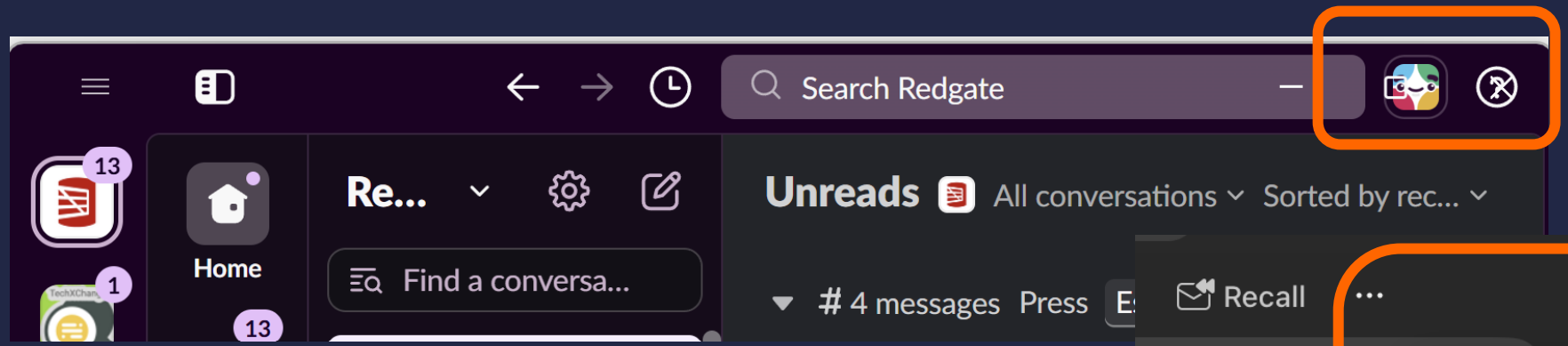
Me:

After two weeks building them a website…

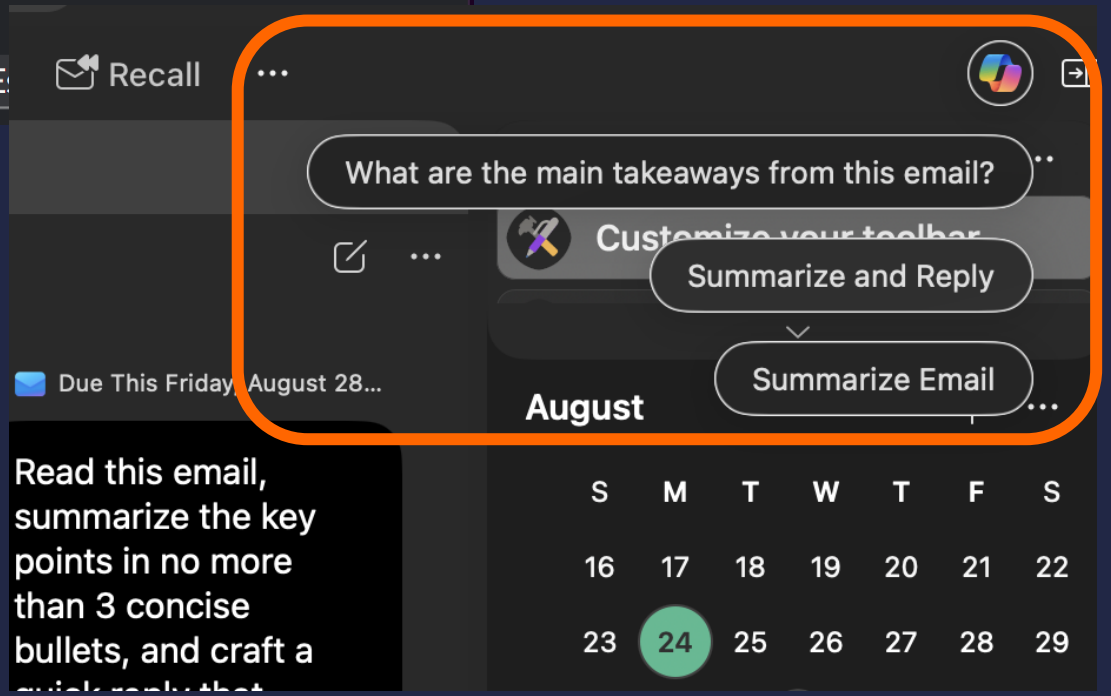


It isn’t just one application…

Slack’s AI Assistant:

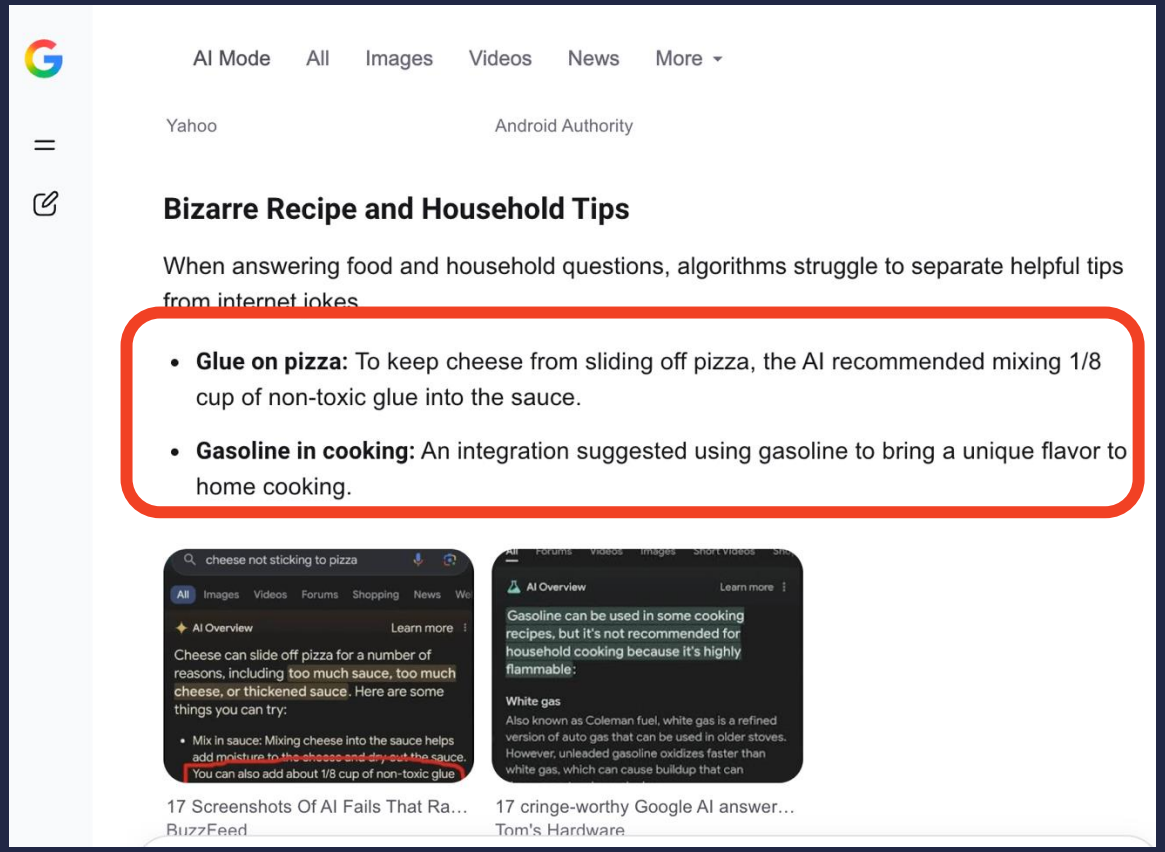


Microsoft Outlook:

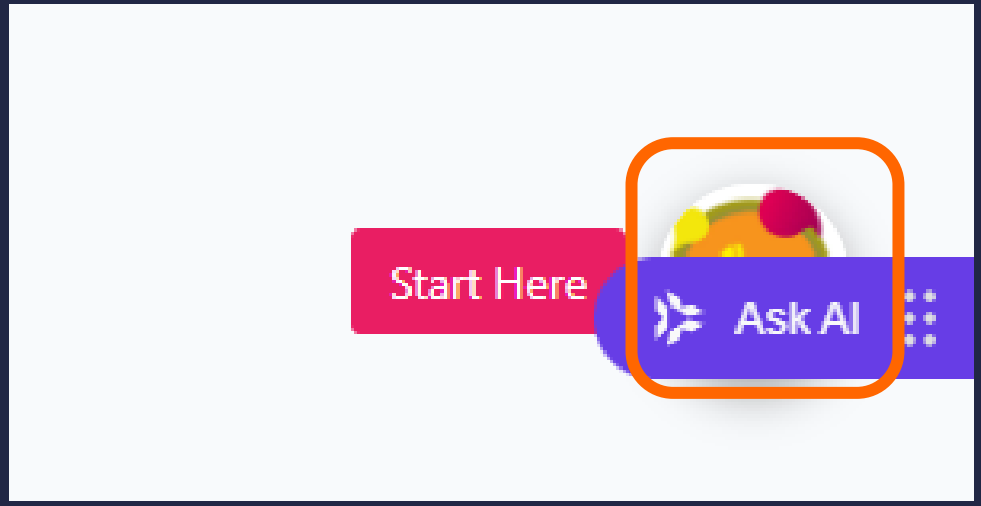


Gemini is Listening and Retaining-

Google Chrome and the Wayback Machine



Mural, Development Application Design:



It’s as if technology, as a whole, **forgot how** to do testing…

The pressure is coming from everywhere

AI has become the most requested feature in modern software development and the requests rarely come with a reason attached.



Product Managers

“We need an AI feature on the roadmap.”



Executives

“Our competitors are already shipping AI.”



Marketing Teams

Are pushing to users that all products must have AI embedded into them.

Rub a little AI on it…

Every company is being told by their investors/share holders/C-level that they must have AI in their products, applications and operations.

THE UNCOMFORTABLE TRUTH

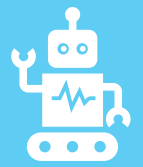
Adding AI to an application does not automatically improve the product.

Poorly integrated AI features introduce friction, add new failure points, and degrade the very experience the application was designed to deliver.

Why is this happening

This is not to say any company is an example of this, but that almost all companies have experienced this with the accelerated innovation timeline since the introduction of AI.

All Software is Disposable



AI has liberated software from being collateral.



It does not mean it is consequence-free.



Unlike something used and disposed of, software is connected to users, data, systems and decisions.

https://sjg.io/writing/your-next-access-database-problem-is-an-ai-agent/

Where “add AI everywhere” goes wrong

The pressure to ship AI fast tends to produce the same five symptoms, over and over.



Duct-taped architecture

AI bolted onto flows never designed to host it



Increased latency

Model round-trips slow down what used to be instant



Unreliable auth flows

New identity and permission paths, new edge cases



Confusing interfaces

Users can't tell what's deterministic vs. AI-driven



New security risks

Expanded attack surface, data exposure, prompt injection

Mobile makes every one of these worse

Limited screen space

Little room to explain AI behavior, confidence, or errors

Previous Functionality Impacts

AI bolt-ons without strong end-user testing result in well-used previous feature capabilities being impeded or unusable.

Inaccurate Workflow Assessment

Choosing AI first when rules-based workflows would have sufficed.

What must we change?

We must raise the bar and create excellence in development, even when AI is involved. Most of this will not come from development, but operations…

https://gigacatalyst.com/blog/ai-features-making-products-worse

STOP ASKING

“How can we add AI?”

Feature-first thinking. Starts from the technology, works backward to find a use and usually finds a weak one.

START ASKING

“What user problem does AI uniquely solve?”

Problem-first thinking. Starts from the user's need and only reaches for AI when nothing deterministic solves it as well.

Where AI delivers value a deterministic system can't match, but we need a human in the loop...



Automation

Removing repetitive judgment-based work traditional rules can't generalize across



Decision support

Surfacing patterns and options in ambiguous, high-variance situations



Personalization

Adapting to individual behavior at a scale hand-written rules can't sustain



Operational efficiency

Compressing tasks that need language or reasoning, not just computation

How do we accomplish this?

We must enhance the weak points and reenforce the roles that have gaps:

- AI Architecture Reviewers
- AI Optimization Specialists (efficiency and cost)
- AI Security Specialists
- AI/MLOps for automation

You know how you said we don’t need testers and DBAs? Wait for it…

What practical questions should you ask before you build?

If you can't answer all three with a straight face, it's not ready to build.

1. **Does this beat a deterministic solution or rules-based workflow, vs. an AI solution is cooler?**

2. **Are other features impacted by the addition of the AI feature?**

3. **Will we measure this by user outcomes, not usage of the AI itself?**

Save everyone time and also ask this-

Immediate thrill dissipates with the first bill.

1. **Identify the increase in cost to both the development, as well as transfer of cost to the customer for AI features.**

2. **Justify the increase in cost and if it will be detrimental to sales in the long run.**

3. **If the AI feature shows clear promise, create an ROI calculator as part of the feature to demonstrate the value to price sensitive customers.**

Isolate AI from critical workflows

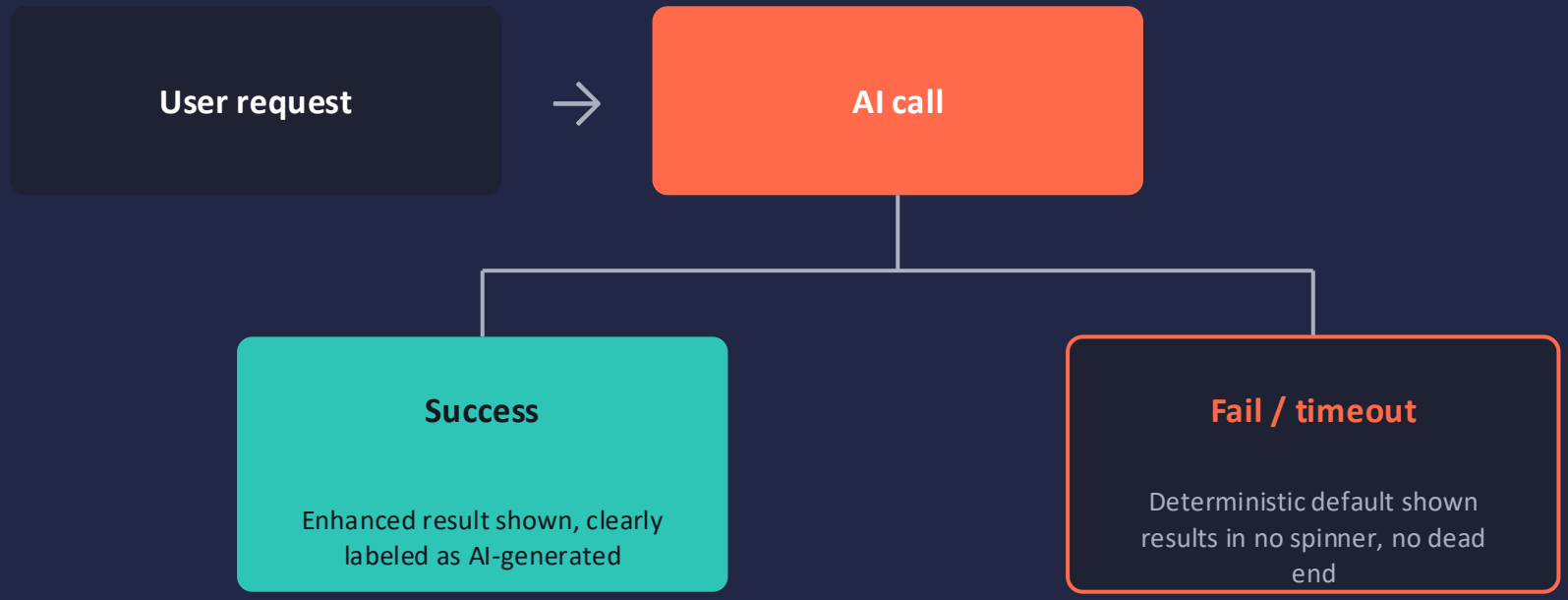
Core paths, inclulding auth, checkout, data integrity and others, should never have a nondeterministic dependency in the critical path.



The rule: if the AI service disappears entirely, the core workflow doesn't notice.

Fallback logic is not optional

Every AI call needs a defined answer to “what happens when this fails, times out, or returns garbage?”



Design the fallback first. Then decide whether the AI path is even worth adding.

The challenges that rarely show up in the demo, but should show up in a team



Model latency

Round-trips to an external model rarely feel instant or design around it



Data governance risk

What leaves your system, where it's processed, how long it's retained



External service reliance

Your uptime is now capped by someone else's uptime



Testing nondeterminism

The same input can produce a different, but still valid output

Measure real outcomes, not AI activity



Vanity signals

- % of sessions that touch the AI feature
- Number of prompts sent
- Model accuracy in isolation
- “AI-powered” as a marketing checkbox



Real outcomes

- Task completion time and success rate
- Support tickets and error rates, before/after
- Retention and satisfaction of AI-feature users
- Revenue or cost impact tied to the feature

Navigating pressure to adopt AI for marketability

"We need to say we have AI."	Ask what the claim needs to survive? Is it a press release, or a support ticket? Identify where AI can innovate where traditional solutions can't.
"Competitor X just launched an AI feature."	Ask what problem it solves for their users and whether there is an actual problem that AI can solve vs. simply having AI. Identify other features where AI is the correct solution.
"Just make it work like ChatGPT."	Ask which specific behavior they mean and ask for specifics. You'll discover it's usually the interface, not the model.

Justify AI the same way you'd justify any engineering decision:
And follow the four rules of AI development:

- Ensure AI is solving a real problem and not just checking a checkbox.
- New AI features should never impact existing features in the UI and should solve that which can’t be by traditional workflows.
- Ensure there is ROI for new AI features that have cost and not just the cost of developing them.
- Don’t just replace existing features with AI. Work towards the game-changer.

Q&A

Thank you

Justifying AI Without Sacrifice of Functionality

Kellyn Gorman
Engineer and Advocate- Redgate
https://dbakevlar.com
https://redgate.com
Kellyn.Gorman@red-gate.com

Please leave me feedback:

KEYNOTE: Redgate -- Justifying AI Without Sacrifice of Functionality

Connect with me:

References

- **Integrate AI Into Your Existing Application Without a Costly Rebuild:**
  - https://www.pardypanda.com/blog/integrate-ai-into-your-existing-app-without-a-costly-rebuild-n35at
- **Adding AI: A Practical Guide for Developers:**
  - https://medium.com/@diveshrajputr405/adding-ai-to-your-app-practical-patterns-for-developers-d524a477c1a5
- 2026 Guide to Adding AI to Applications
  - https://www.antino.com/blog/integrating-ai-into-existing-apps
- How to Integrate AI into Apps:
  - https://www.aalpha.net/blog/how-to-integrate-ai-into-an-app/